

TP - 03. Python - Scanner de port

Propriétés (4)	Valeurs
tags	B3, python
version	0.1
Bloc	Bloc3
Compétences du référentiel	B3.5.4 - SLAM - Prévenir les attaques

Rappels

-  [Memo - TCP IP - Liste des ports TCP UDP](#)

Objectifs

- Un scanner de port est utilisé pour vérifier **quels ports sont ouverts** et prêts à recevoir des connexions sur un hôte distant

Attention

Il est important de noter que l'utilisation de scanners de ports sur des systèmes distants sans autorisation préalable peut être contraire à la loi. Vous devez toujours obtenir une autorisation explicite avant de scanner des ports sur un réseau qui ne vous appartient pas.

La **tentative** d'accès frauduleux à un système informatique est expressément incriminée par l'**article 323-7** du **Code pénal** français. Cette disposition prévoit que la tentative est **punie des mêmes peines** que l'infraction consommée, permettant ainsi de réprimer les actes préparatoires même si l'accès

effectif n'a pas été obtenu.

🔗 Travail à faire

Dans un dossier **TP-Python-Port-Scanner**, créer un fichier **scanner.py**

Suivez l'étapes de ce TP pour remplir ce fichier.

Dépendances

```
import socket
import threading
from queue import Queue
```

On commence par importer les modules dont on va avoir besoin dans le script.

☒ **Socket**

Ce module est utilisé pour effectuer des opérations réseau, notamment pour établir des connexions sur les ports.

☒ **Threading**

Ce module permet de créer et de gérer des threads (processus légers) qui effectueront le balayage des ports de manière parallèle.

<https://datascientist.fr/blog/tutoriel-python-introduction-au-threading>

☒ **Queue**

Cette classe est utilisée pour gérer une file d'attente (queue) qui contiendra les ports à scanner.

Déclaration des variables

L'adresse IP de la cible et le stockage des ports ouverts

```
# Demande à l'utilisateur
target = input("Entrer l'adresse IP de la cible : «")

# Création de la queue
queue = Queue()

# Liste des ports ouverts
open_ports = []
```

Nous allons créer une variable qui va *demande* à l'utilisateur de saisir l'adresse IP de la cible.
Cette adresse IP sera ensuite **scannée** pour déterminer quels ports sont ouverts.

Ensuite une file d'attente (**queue**) est créée pour stocker les numéros de port à scanner et une liste vide **open_ports** est créée pour stocker les numéros de port ouverts.

La fonction port_scan

```
def port_scan(port):
    try:
        # Configuration de socket
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        # Connexion à la cible sur le port passé en paramètre
        sock.connect((target, port))
        return True
    except:
        return False
```

Objectif de la fonction port_scan:

- Prend en paramètre un numéro de port
 - On pourrait vérifier qu'il s'agit bien d'un entier compris entre 1 et 65535
 - Tentative de connexion au port passé en paramètre
 - Renvoie **True** en cas de réussite
 - Renvoie **False** en cas d'échec (*port fermé ou inaccessible*)
-

Les fonctions `fill_queue` et `executor`

```
def fill_queue(port_list):  
    for port in port_list:  
        queue.put(port)  
  
def executor():  
    while not queue.empty():  
        port = queue.get()  
        if port_scan(port):  
            print(« Le port {} est ouvert ».format(port))  
            open_ports.append(port)
```

Objectif de la fonction `fill_queue`:

- Prend en paramètre une liste de ports
 - On pourrait vérifier qu'il s'agit bien d'entiers compris entre 1 et 65535
- Parcours de la liste
- Pour chaque élément de la liste, ajout à la **queue**

Objectif de la fonction `executor`:

- Parcours de la **queue** jusqu'au bout (**empty**)
 - Pour chaque élément de la **queue**
 - Récupération du port
 - Appel de la fonction **port_scan** en passant en paramètre le port
 - Si la fonction **port_scan** renvoie **true**
 - Affichage du texte *Le port XX est ouvert*
 - Ajout du port en cours dans le tableau **open_ports**
-

Création de la liste de ports à scanner et remplissage de la liste d'attente

```
# Liste des ports de 1 à 1024
port_list = range(1,1024)
# Appel de la fonction fill_queue
fill_queue(port_list)
# Stockage des threads dans une liste
thread_list = []
```

Les numéros de port de la liste **port_list** sont ajoutés à la file d'attente en appelant la fonction **fill_queue(port_list)**.

Une liste vide **thread_list** est créée pour stocker les **threads**.

Démarrage des threads et affichage des ports

```
for t in range(500):
    # Définition de la fonction exécutée par le thread
    thread = threading.Thread(target=executor)
    # Ajout du thread à thread_list
    thread_list.append(thread)

for thread in thread_list:
    # Lancement du thread
    thread.start()

for thread in thread_list:
    # Attend que le thread soit terminé
    thread.join()

print("Les ports ouverts sont : ", open_ports)
```

- Un total de **500 threads** est créé, chacun exécutant la fonction **executor()**.
 - Chaque thread est démarré en appelant **thread.start()**.
 - Les threads attendent pour s'assurer que tous les threads ont terminé leur exécution avant de continuer.
 - Une fois que tous les threads sont terminés, le programme affiche la liste des ports ouverts dans la variable **open_ports**.
-

Travail à faire

🔧 Travail à faire

- Reprenez le code proposé ci-dessus et rassemblez le dans un script python nommé **scanner.py**
- Exécutez le et ciblez votre propre machine
- Quels sont les résultats qui apparaissent?
- Tester le sur la VM de votre camarade
- Créez un **dépôt** GitHub dans lequel vous enverrez ce script ainsi qu'un fichier README.md

A voir

- 🔧 [NMAP - Network Mapper](#)